Contents

Day 1 — The New AI Application Stack

Traditional software engineering begins with deterministic computation.

You write a function:

```
result = process(input)
```

and expect the same input to produce the same result.

AI applications introduce a fundamentally different computational primitive:

```
input
  ↓
foundation model
  ↓
probabilistic output
```

The model is not simply another library.

It is a component whose behavior depends on:

- model version
- prompt
- context
- tokenization
- inference parameters
- available tools
- modality
- latency
- capacity
- provider
- cost
- and the statistical behavior of the underlying model

This changes the nature of application engineering.

The central idea of Day 1 is:

> **Treat the LLM as an engineering component, not as an oracle.**

Once you adopt that mindset, model selection, prompting, structured output, tool calling, token accounting, latency measurement, and evaluation become engineering concerns rather than prompt-writing tricks.

The first project of the bootcamp is therefore intentionally small:

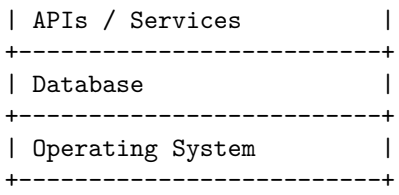
> **Build a model playground that lets you inspect and compare LLMs as computational components.**

---

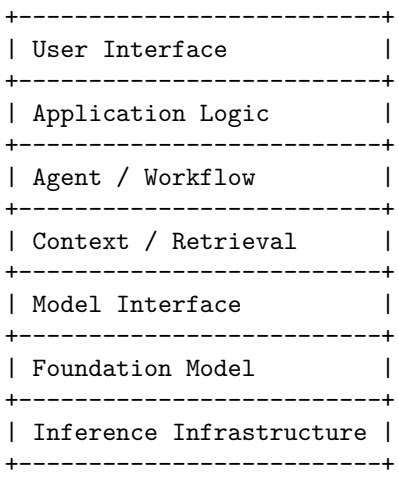
1. The New Application Stack

A conventional application stack might look like:

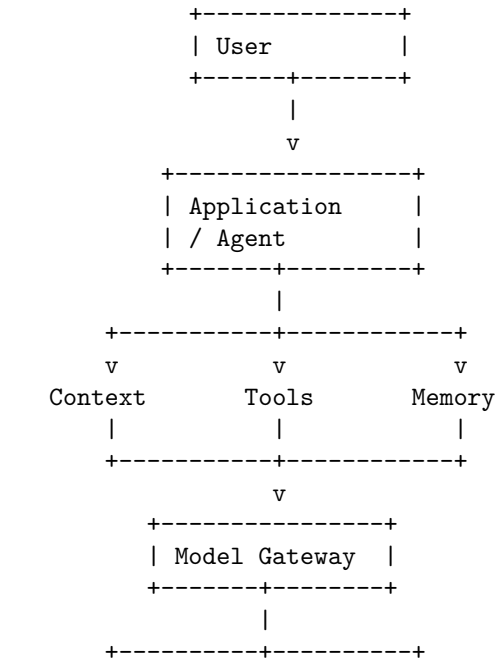
```
+--------------------------+
| UI                       |
+--------------------------+
| Application Logic        |
+--------------------------+
```



An AI application adds a new computational layer:



And increasingly:



```
      v           v           v
Model A     Model B     Local Model
```

The foundation model is therefore one component in a larger computational system.

That distinction will become increasingly important throughout the bootcamp.

---

2. Foundation Models

A foundation model is a large pretrained model capable of performing many downstream tasks.

For language applications, the model typically operates over tokens and learns statistical relationships between them.

At a simplified level:

```
prompt
  ↓
tokenization
  ↓
token sequence
  ↓
neural network
  ↓
probability distribution
  ↓
next token
  ↓
repeat
  ↓
output
```

The model does not directly manipulate "meaning" in the same way a traditional program manipulates structured variables.

It operates on representations learned during training.

That gives it extraordinary flexibility.

The same model can potentially:

- summarize documents
- write code
- classify text
- extract structured information
- reason through problems

- call tools
- analyze images
- transform text
- generate plans

But that flexibility comes with an engineering cost:

> **The model's interface is probabilistic.**

The application must therefore constrain, observe, and evaluate its behavior.

---

3. Models Are Components With Contracts

An engineer does not treat a database as:

> "A magical system that usually returns useful information."

You reason about:

- schema
- latency
- consistency
- availability
- errors
- capacity
- cost

The same mindset should apply to models.

For every model, ask:

```
What does it accept?
What does it return?
What does it do well?
Where does it fail?
How much does it cost?
How fast is it?
What context can it handle?
What modalities does it support?
How deterministic is it?
How reliable are its structured outputs?
```

This is the beginning of **model engineering**.

---

4. APIs Versus Local Models

One of the first architectural decisions is whether inference happens through a remote API or locally.

API-based inference

```
Application
     |
     | HTTPS
     v
Model Provider
     |
     v
GPU infrastructure
     |
     v
Model
```

Advantages include:

- access to frontier models
- no model-serving infrastructure
- elastic capacity
- rapid model upgrades
- potentially high quality

Costs include:

- network latency
- per-token pricing
- provider dependency
- data-governance considerations
- rate limits
- external availability

Local inference

```
Application
     |
     v
Local inference runtime
     |
     v
GPU / accelerator
     |
     v
Model
```

Advantages include:

- local data processing
- predictable infrastructure
- potentially lower marginal cost
- offline operation
- greater control over model versions

Costs include:

- hardware
- memory requirements
- model management
- serving infrastructure
- optimization
- operational responsibility

Neither approach is universally better.

The correct question is:

> **What deployment model satisfies the application's quality, privacy, latency, cost, and operational requirements?**

---

5. The Model Gateway

Even a small AI application benefits from thinking in terms of a model abstraction.

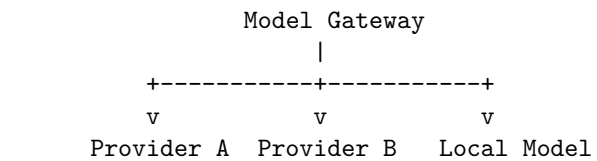
Instead of scattering provider-specific calls throughout the application:

```
client.openai.chat.completions.create(...)
```

use an application-level interface:

```
response = model.generate(
    messages=messages,
    parameters=params
)
```

Then the implementation can route to:



This creates several future possibilities:

- model A for high-quality requests
- model B for cheap requests
- local model for sensitive data
- fallback model during provider outages
- specialized model for vision
- smaller model for classification

The model gateway becomes an architectural control point.

---

6. Tokens

LLMs generally do not consume raw strings directly.

Text is converted into tokens.

For example:

```
"Architecture matters."
```

might become a sequence resembling:

```
["Architecture", " matters", "."]
```

The exact tokenization depends on the model.

This matters because many properties of an AI application are expressed in tokens:

```
context size
input cost
output cost
throughput
latency
rate limits
```

Instead of thinking:

```
request size = 4 KB
```

AI engineers frequently need to think:

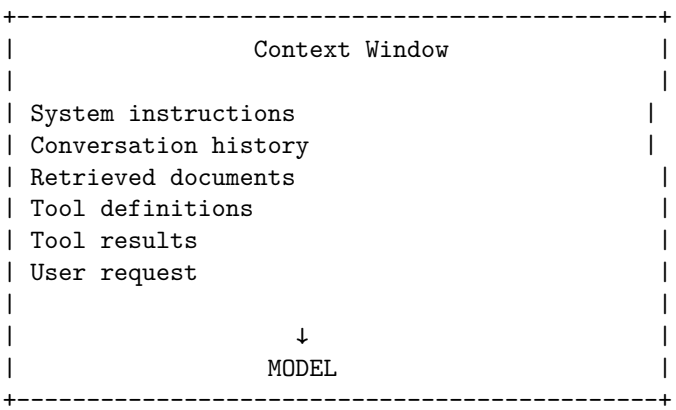
```
input = 3,200 tokens
output = 850 tokens
total = 4,050 tokens
```

---

7. Context Windows

The model operates within a context window.

Conceptually:



The context window is a computational resource.

It has consequences for:

- memory
- latency
- cost
- retrieval strategy
- prompt construction
- application architecture

More context is not necessarily better.

A large amount of irrelevant context can make the system less reliable.

This will become a major theme in Day 2.

---

8. Prompting Is Interface Design

Prompting is often treated as an informal art.

A better engineering perspective is:

> **A prompt is an interface specification for a probabilistic component.**

A useful prompt may define:

```
Role
Goal
Constraints
Available context
Expected behavior
```

```
Output format
Failure behavior
```

For example:

```
You are a document extraction system.

Extract:
- customer_name
- invoice_number
- total_amount

Rules:
- Do not infer missing values.
- Use null when a field is absent.
- Return valid JSON matching the schema.
```

The prompt is effectively part of the program.

But unlike conventional code, its semantics are probabilistic.

That means prompts must eventually be tested and evaluated.

---

9. Structured Outputs

One of the most important techniques in production AI engineering is constraining model output.

A fragile application might ask:

```
Return the customer's name and age.
```

and receive:

```
Sure! The customer appears to be Robert, who is approximately 53 years old.
```

That may be acceptable for a chat interface.

It is problematic if downstream software expects:

```
{
  "name": "Robert",
  "age": 53
}
```

Structured output provides an explicit contract.

For example:

```python
class Customer(BaseModel):
    name: str
    age: int
```

The application can then require the model to produce output conforming to that schema.

Conceptually:

```
LLM
 |
 v
Structured output constraint
 |
 v
Schema validation
 |
 +-- valid ---> application
 |
 +-- invalid -> retry / repair / failure
```

This is one of the first places where probabilistic computation is transformed into a more deterministic interface.

---

10. Tool Calling

An LLM by itself cannot directly perform arbitrary external operations.

A tool interface allows the model to request an operation:

```
Model
  |
  v
Tool call
  |
  +-- search()
  +-- database_query()
  +-- calculator()
  +-- get_weather()
```

For example:

```
{
  "tool": "search",
  "arguments": {
    "query": "latest AI architecture research"
```

```
  }
}
```

The application executes the tool.

Then:

```
Tool
  |
  v
Result
  |
  v
Model
```

This creates a fundamental architectural distinction:

> **The model decides what it wants to do; the application decides whether and how that action is actually performed.**

This distinction becomes extremely important for security.

Never assume that because a model requested a tool call, the tool should automatically execute it.

The application should enforce:

- permissions
- argument validation
- rate limits
- authentication
- authorization
- sandboxing
- resource limits

Tool calling therefore turns an LLM from a text generator into a component of an executable system.

---

11. Multimodal Models

Modern foundation models increasingly operate across multiple modalities.

Instead of:

```
text → text
```

we can have:

```
text  -----+
image -----+
```

```
audio -----+--> Model ---> text
video -----+
```

This changes application architecture.

For example, a document assistant may need to process:

```
PDF
 |
 +-- text
 +-- tables
 +-- diagrams
 +-- images
```

A text-only pipeline may lose important information.

A multimodal model can potentially reason over several representations simultaneously.

But multimodality introduces additional engineering questions:

- input size
- preprocessing
- modality-specific tokenization
- latency
- cost
- output reliability
- modality-specific evaluation

The engineering principle remains unchanged:

> **Treat each model capability as a measurable component with a defined contract.**

---

12. Inference Parameters

Models expose parameters that affect generation behavior.

Common examples include:

- temperature
- top-p
- maximum output tokens
- stop sequences
- seed, where supported
- reasoning or effort controls, where supported

These parameters influence the output distribution.

A simplified intuition for temperature is:

```
Low temperature
    ↓
more concentrated probability distribution
    ↓
more predictable output

High temperature
    ↓
flatter probability distribution
    ↓
more varied output
```

But temperature should not be treated as a universal "creativity slider."

Different models and inference systems may behave differently.

The engineering lesson is:

> **Inference parameters are part of the system configuration and should be measured, versioned, and evaluated.**

If changing temperature changes application behavior, temperature is effectively part of the application specification.

---

13. Determinism Is Not Binary

A common misconception is:

> "Set temperature to zero and the model becomes deterministic."

In practice, reproducibility can depend on:

- model version
- provider implementation
- sampling implementation
- hardware
- backend changes
- tool results
- retrieval results
- system prompts
- context
- inference configuration

Even when generation is highly constrained, the surrounding system may remain nondeterministic.

For example:

```
User
 ↓
Retrieval
 ↓
Model
 ↓
Tool
 ↓
Model
```

If retrieval changes, the final output may change even when model parameters remain constant.

This is why AI systems must be evaluated at the **system level**, not only the model level.

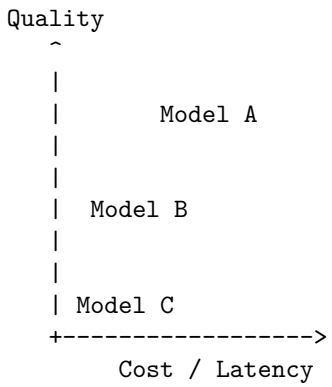
---

14. Model Selection

There is rarely one universally best model.

Model selection is a multi-objective optimization problem.

Consider:



A model can be:

- more accurate
- faster
- cheaper
- better at coding
- better at vision
- better at structured extraction
- better at tool calling

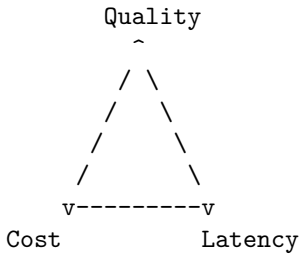
These properties may conflict.

The correct model therefore depends on the workload.

---

15. The Quality-Cost-Latency Triangle

A useful first approximation is:



In reality, there are many more dimensions.

For example:

```
Quality
Latency
Cost
Context capacity
Tool reliability
Structured output reliability
Reasoning capability
Multimodal capability
Availability
Privacy
Deployment control
```

Model selection is therefore an architectural decision.

Not a benchmark leaderboard decision.

---

16. Measure Before Choosing

Suppose you have three candidate models:

Model	Quality	Latency	Cost
A	95	1.8 s	$0.020
B	91	0.8 s	$0.006

Model	Quality	Latency	Cost
C	84	0.3 s	$0.001

Which is best?

There is no answer without knowing the application.

For a medical research assistant where correctness dominates:

`Model A`

may be appropriate.

For a high-volume classification pipeline:

`Model C`

might be dramatically better economically.

For an interactive application where users expect immediate responses:

`Model B`

could dominate.

The engineering process is:

```
Requirements
    ↓
Workload
    ↓
Evaluation dataset
    ↓
Benchmark models
    ↓
Cost / latency / quality analysis
    ↓
Model selection
```

This is much stronger than:

> "Everyone says Model A is the best."

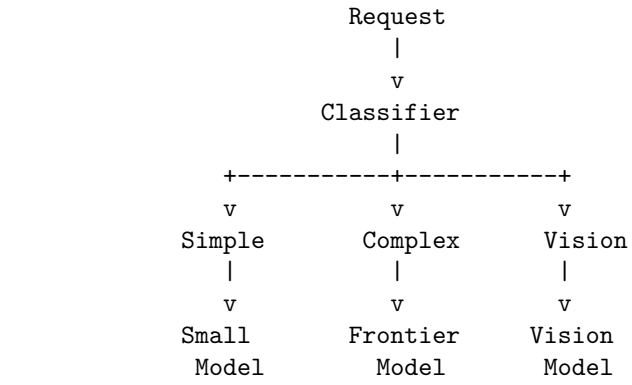
---

17. Model Routing

Once you think in terms of models as components, another possibility appears:

> **Why use the same model for every request?**

Consider:



Simple requests can use inexpensive models.

Complex requests can use more capable models.

This is model routing.

The economic impact can be substantial at scale.

---

18. Latency Is a System Property

Model latency is not the same thing as application latency.

Consider:

```
Request
 |
 +-- authentication      20 ms
 +-- retrieval           80 ms
 +-- reranking           60 ms
 +-- model              900 ms
 +-- tool call          300 ms
 +-- persistence         30 ms
                         -----
                         1.39 s
```

The model accounts for most of the latency.

But not all of it.

Now consider an agent:

```
Model call 1
   ↓
Tool call
   ↓
```

```
Model call 2
   ↓
Tool call
   ↓
Model call 3
```

Even if each model call takes only 500 ms:

```
3 x 500 ms = 1.5 seconds
```

before tool latency is included.

Architecture therefore determines latency.

---

19. Streaming

Streaming changes the user-perceived latency profile.

Without streaming:

```
Request
   |
   +--------------- 2 seconds --------------->
                                              Response
```

With streaming:

```
Request
   |
   +-- 300 ms --> first token
   +-- 400 ms --> more tokens
   +-- 500 ms --> more tokens
   +-- 2 sec  --> complete response
```

The total generation time may not change.

But **time to first token** can improve perceived responsiveness.

Therefore a model playground should measure both:

```
TTFT = time to first token
TTLB = time to last byte/token
```

These are different metrics.

---

20. Cost Is a First-Class Metric

For API models, cost is often approximately related to:

```
input tokens x input price
+
output tokens x output price
```

A request might therefore be represented as:

```
Input:
4,000 tokens

Output:
1,000 tokens

Cost:
$X
```

But AI application cost is broader than model tokens.

Consider:

```
Model inference
+
embedding generation
+
vector search
+
database
+
object storage
+
network
+
compute
+
observability
```

At scale, these costs interact.

For example:

```
More context
   ↓
More input tokens
   ↓
Higher cost
   ↓
Higher latency
   ↓
Lower throughput
```

A seemingly harmless prompt design decision can therefore become an architectural cost problem.

---

21. The Model Playground

The first mini-project should be deliberately small.

Do not build a chatbot.

Build an **instrumented model laboratory**.

The application should provide a common interface to several models.

Conceptually:

```
+-----------------------------------------+
|              Model Playground           |
+-----------------------------------------+
| Prompt                                  |
| Model                                   |
| Parameters                              |
| Structured schema                       |
+-----------------------------------------+
| Run                                     |
+-----------------------------------------+
| Output                                  |
|                                         |
| Metrics                                 |
|   TTFT                                  |
|   Total latency                         |
|   Input tokens                          |
|   Output tokens                         |
|   Estimated cost                        |
+-----------------------------------------+
```

The interface should make models comparable.

---

22. Required Capabilities

The playground should support:

Multiple models

For example:

```
Model A
Model B
Model C
Local Model
```

The exact providers are less important than having genuinely different inference characteristics.

Streaming

Display tokens as they arrive.

Measure:

```
time to first token
total generation time
tokens/sec
```

Token accounting

Track:

```
input tokens
output tokens
total tokens
```

Cost estimation

Given provider pricing:

```
cost =
input_tokens x input_price
+
output_tokens x output_price
```

Store the pricing information separately from application logic so it can be updated.

Output comparison

Given the same prompt:

```
Prompt
  |
  +--> Model A
  +--> Model B
  +--> Model C
```

display the outputs side by side.

Structured output

Allow the user to specify a schema:

```
{
  "summary": "string",
  "confidence": "number",
  "topics": ["string"]
}
```

and require the model to produce valid output.

---

23. Instrumentation

Do not merely display the answer.

Capture metadata.

For every request, record something like:

```
{
  "model": "model-x",
  "timestamp": "...",
  "input_tokens": 1432,
  "output_tokens": 327,
  "time_to_first_token_ms": 412,
  "total_latency_ms": 1834,
  "tokens_per_second": 178,
  "estimated_cost": 0.0124,
  "structured_output_valid": true
}
```

This transforms the playground from a chatbot into an experimental instrument.

That distinction matters.

You are not trying to determine:

> "Which answer sounds best?"

You are trying to understand:

> **"How does each model behave as a computational component?"**

---

24. Build a Common Model Interface

A useful abstraction might look conceptually like:

```python
class Model:
    def generate(
        self,
        messages,
        *,
        parameters=None,
        schema=None,
        stream=False,
    ):
        ...
```

Every provider implements the same conceptual contract.

Then the benchmark harness can remain provider-independent:

```
Benchmark
    |
    +-- Model A
    +-- Model B
    +-- Model C
    +-- Local Model
```

This is the first practical application of the architecture concepts from Day 8.

You are deliberately separating:

```
application
    ↓
model abstraction
    ↓
provider implementation
```

---

25. Make the Playground a Scientific Instrument

A good benchmark is reproducible.

Use a fixed collection of prompts.

For example:

```
Prompt 1 - summarization
Prompt 2 - extraction
Prompt 3 - classification
Prompt 4 - reasoning
Prompt 5 - coding
```

```
Prompt 6 - structured output
Prompt 7 - long context
Prompt 8 - tool calling
```

Then run every model against the same workload.

Record:

```
Model
Prompt
Configuration
Input tokens
Output tokens
Latency
TTFT
Cost
Output
Validation result
```

Now you have experimental data.

---

26. Compare Outputs Carefully

Output comparison is more difficult than it first appears.

For deterministic tasks:

```
Expected:
42
```

comparison is easy.

For open-ended generation:

```
Explain why this architecture fails.
```

there may be many valid answers.

You may need:

- rubric-based evaluation
- human comparison
- LLM-as-judge
- task-specific metrics

This is the beginning of evaluation engineering.

For Day 1, however, simply preserve the outputs and measurements.

Later chapters will build the evaluation machinery around them.

---

27. Structured Output as a Boundary

One particularly important experiment is to compare:

Unconstrained output

```
Prompt
  ↓
Model
  ↓
Free-form text
```

with:

Structured output

```
Prompt
  ↓
Model
  ↓
Schema constraint
  ↓
Validation
  ↓
Typed object
```

Measure:

```
schema success rate
retry rate
latency
output tokens
```

You may discover that a model that sounds excellent in natural language performs poorly as a structured data generator.

That is an important engineering discovery.

Model quality is task-dependent.

---

28. Failure Modes to Explore

Do not build the playground and only test successful cases.

Try to break it.

Invalid JSON

Ask for a strict schema and see whether the model complies.

Very long prompts

Observe:

- latency
- cost
- degradation
- context limits

Large outputs

Observe:

- generation speed
- truncation
- cost

Streaming failures

What happens if the connection breaks halfway through generation?

Provider failure

What happens if the API returns an error?

Rate limiting

What happens when the provider returns a quota error?

Timeout

What happens if inference takes too long?

Model mismatch

What happens if one model does not support a requested capability?

These failures will become important later when you build production systems.

---

29. The First Architectural Lesson

At the end of this exercise, you should have discovered something subtle.

The model call itself is small:

```
response = model.generate(...)
```

But everything around it is engineering.

You need:

```
model selection
configuration
authentication
timeouts
retries
streaming
token accounting
cost accounting
structured output
validation
observability
error handling
fallbacks
evaluation
```

This is the central transition of the course.

The hard part is not:

> "How do I call an LLM?"

The hard part is:

> **"How do I incorporate an LLM into a reliable computational system?"**

---

30. From Model API to AI System

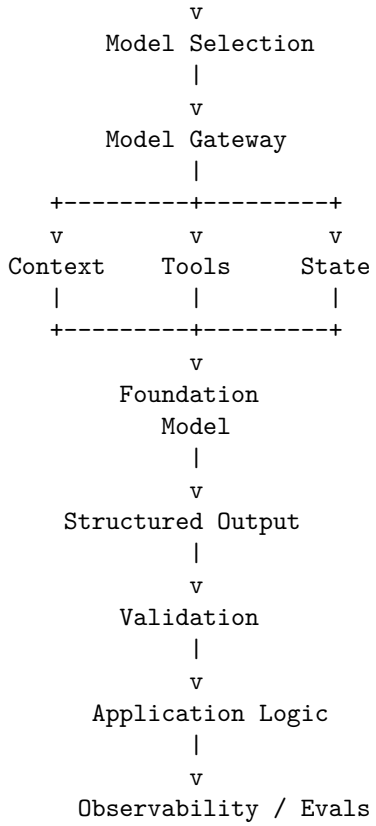
The conceptual progression is:

```
2022-style mental model


Prompt
  ↓
LLM
  ↓
Answer
```

The engineering mental model is:

```
                  +---------------+
                  | Requirements  |
                  +-------+-------+
```



This is the beginning of AI systems engineering.

---

31. Key Takeaways

1. **An LLM is an engineering component, not an oracle.**
2. **Foundation models introduce probabilistic computation into the application stack.**
3. **Model APIs should be treated as interfaces with measurable contracts.**
4. **Remote and local inference are architectural alternatives with different quality, cost, latency, privacy, and operational tradeoffs.**
5. **Tokens are a fundamental resource.** They affect context capacity, latency, throughput, and cost.
6. **Context is a computational resource.** More context is not automati-

cally better.

7. **Prompting is interface design.** Prompts should eventually become versioned, tested, and evaluated artifacts.
8. **Structured outputs convert probabilistic generation into a more reliable interface between the model and application code.**
9. **Tool calling turns an LLM into a component capable of participating in executable workflows.** The application must remain responsible for authorization and execution.
10. **Multimodal models expand the input/output space but also introduce new latency, cost, preprocessing, and evaluation requirements.**
11. **Inference parameters are part of system configuration.** They should be measurable and reproducible.
12. **There is no universally best model.** Model selection is a workload-specific optimization across quality, latency, cost, capability, and operational constraints.
13. **Streaming changes perceived latency even when total generation time is unchanged.**
14. **Instrumentation is essential.** Measure TTFT, total latency, token counts, throughput, cost, and structured-output validity.
15. **A model playground should be an experimental instrument, not merely a chatbot.**
16. **The same prompt should be evaluated across multiple models under controlled conditions.**
17. **The first AI engineering skill is not prompt writing. It is learning to reason about the model as a component inside a larger system.**

The fundamental mental shift is:

```
Don't think:

    "I have an AI that answers questions."

Think:

    "I have a probabilistic computational component
     with a measurable interface, resource requirements,
     failure modes, costs, and quality characteristics."
```

Once you think this way, the rest of the discipline follows naturally.

Tomorrow, the model itself becomes only one part of the problem.

The next question is:

> **What information should the model receive, how should that information be constructed, and what happens when the context becomes larger than the model can reliably use?**